MANGALORE INSTITUTE OF TECHNOLOGY & ENGINEERING

(A Unit of Rajalaxmi Education Trust ®, Mangalore)
Autonomous Institute Affiliated to V.T.U, Belagavi, Approved by AICTE, New Delhi
Accredited by NAAC with A+ Grade & ISO 9001:2015 Certified Institution



PROJECT REPORT

23AISE409 PROJECT PHASE – II

"HOMIE: MACHINE LEARNING-BASED TRANSACTION RISK ASSESSMENT ON SOLANA"

Submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF ENGINEERING

in

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

by

Bharath Kumar Shetty	4MT23AI013
Deepthi P. Salian	4MT23AI017
Shreya Shetty	4MT23AI051
Swaroop Nellitheertha	4MT23AI056

Under the Guidance of
Mr. Manjunath Prasad

DEPARTMENT OF
ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

2026–27

MANGALORE INSTITUTE OF TECHNOLOGY & ENGINEERING

(A Unit of Rajalaxmi Education Trust ®, Mangalore)

Autonomous Institute Affiliated to V.T.U, Belagavi, Approved by AICTE, New Delhi

Accredited by NAAC with A+ Grade & ISO 9001:2015 Certified Institution

MITE

Invent Solutions

DEPARTMENT OF

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

CERTIFICATE

This is to certify that the Project Phase – II (**23AISE409**) work entitled **"HOMIE: Machine Learning-Based Transaction Risk Assessment on Solana"** is the project work carried out by the following students in partial fulfilment of the requirements for the award of the Bachelor of Engineering degree in Artificial Intelligence and Machine Learning at Mangalore Institute of Technology & Engineering (Autonomous), affiliated to Visvesvaraya Technological University, Belagavi, during the academic year 2026–27.

Bharath Kumar Shetty	4MT23AI013
Deepthi P. Salian	4MT23AI017
Shreya Shetty	4MT23AI051
Swaroop Nellitheertha	4MT23AI056

The report is submitted for examination under the guidance of **Mr. Manjunath Prasad**.

Mr. Manjunath Prasad	Head of Department
Project Guide	AI&ML

Principal

DECLARATION

We declare that the Project Phase – II (23AISE409) report entitled **"HOMIE: Machine Learning-Based Transaction Risk Assessment on Solana"**, submitted in partial fulfilment of the requirements for the Bachelor of Engineering degree in Artificial Intelligence and Machine Learning, presents our project work carried out under the guidance of **Mr. Manjunath Prasad**. Sources of data, software and published research used in the work are acknowledged in the report. This report has not been submitted by us for the award of another degree.

Sr. No.	**Name and USN**	**Signature**
1	Bharath Kumar Shetty (4MT23AI013)	
2	Deepthi P. Salian (4MT23AI017)	
3	Shreya Shetty (4MT23AI051)	
4	Swaroop Nellitheertha (4MT23AI056)	

Place: Moodbidri

Date: _____

ACKNOWLEDGEMENTS

We express our gratitude to our project guide, Mr. Manjunath Prasad, for guidance during Project Phase – II. We thank the Head of Department, Principal and management of Mangalore Institute of Technology & Engineering for providing the academic environment for this work. We also thank the faculty of the Department of Artificial Intelligence and Machine Learning for their support.

We acknowledge the maintainers of SolArchive for making blockchain records publicly accessible, and the developers of Solana, scikit-learn and the open-source tools used in the implementation. We thank our families and classmates for their encouragement and support.

Bharath Kumar Shetty (4MT23AI013)
Deepthi P. Salian (4MT23AI017)
Shreya Shetty (4MT23AI051)
Swaroop Nellitheertha (4MT23AI056)

ABSTRACT

HOMIE assesses Solana transaction execution risk when the signer has no available outcome history. It constructs chronological features from public SolArchive records and compares histogram gradient boosting models with and without non-signer writable-account history. A one-hour outcome delay restricts features to earlier available labels. Both models also receive a learned identity score in the stronger comparison. On 17,904 reserved first-appearance transactions, average precision increases from 0.1688 to 0.2004; a further test on 19,515 transactions increases it from 0.2207 to 0.2393. The application supports historical replay, fitted-model inference, live transfer checks and decision-to-receipt recording. A local-validator experiment verifies that a landed execution failure can incur a fee while normal preflight can reject the same error without a fee. The results demonstrate additional historical failure-ranking signal; measured ML-driven fee savings are not claimed.

Keywords: Solana, histogram gradient boosting, account history, transaction failure, temporal evaluation.

TABLE OF CONTENTS

1 INTRODUCTION **1**

1.1 Background 1

1.2 Problem Statement 2

1.3 Motivation / Need for the Project 2

1.4 Objectives 2

1.5 Scope of the Project 2

1.6 Significance and Expected Contribution 3

1.7 Organization of the Report 3

2 LITERATURE REVIEW **4**

2.1 Introduction 4

2.2 Review of Existing Work 4

2.3 Comparative Analysis 6

2.4 Research and Design Gap 7

2.5 Summary 7

3 PROPOSED SYSTEM AND SCOPE **8**

3.1 Problem Formulation 8

3.2 Proposed Solution 8

3.3 Requirements 8

3.4 System Architecture / Block Diagram 9

3.5 Functional / System Components 11

3.6 Scope and Constraints 11

4 METHODOLOGY **13**

4.1 Objective of the Study 13

4.2 Method / Approach 13

4.3 Measurement / Evaluation Parameters 13

4.4 Tools, Equipment and Resources 14

4.5 Experimental Procedure 14

4.6 Model Architecture 15

4.7 Error Sources and Mitigation 17
4.8 Data Processing and Analysis 17

5 IMPLEMENTATION AND EXPERIMENTAL WORK 19
5.1 Implementation Environment 19
5.2 Module-wise Implementation 19
5.3 Experimental Setup and Test Cases 21
5.4 Prototype / System Implementation 22
5.5 Testing and Validation 22

6 RESULTS, ANALYSIS AND DISCUSSION 24
6.1 Results 24
6.2 Analysis 25
6.3 Discussion 25
6.4 Comparison with Existing Approaches 26
6.5 Significance and Implications 26
6.6 Potential Applications 26

7 CONCLUSION AND FUTURE SCOPE 27
7.1 Summary of Work 27
7.2 Conclusions 27
7.3 Contributions 27
7.4 Utility / Applications 27
7.5 Limitations 27
7.6 Scope for Future Work 28

References 29

LIST OF FIGURES

3.1 Implemented research and application architecture. 10

3.2 Live checking and outcome-recording workflow. 10

4.1 Historical research data flow. 15

5.1 Checks before a receipt becomes an observed outcome. 21

6.1 Identity-controlled first-appearance AP on two test dates. The plot shows ranking performance, not classification accuracy. 24

LIST OF TABLES

2.1 Relationship between existing resources and HOMIE 4
2.2 Methodological literature and its use in the project 6

3.1 Implemented technology stack 9
3.2 Implemented modules and responsibilities 11
3.3 Users and supported activities 11

4.1 Chronological experimental windows (2025) 13
4.2 Histogram gradient boosting configuration 15
4.3 Feature groups and their interpretation 18

5.1 Study-store entities 20
5.2 Test inputs and expected safeguards 21
5.3 Implemented checks and recorded verification 23

6.1 Cold-history AP without the identity-score control 24
6.2 First-appearance results with the identity-score control 24
6.3 Measured local fee-mechanism outcomes 25

ABBREVIATIONS / NOTATIONS

Abbreviation	Meaning
AI	Artificial intelligence
ML	Machine learning
HGB	Histogram gradient boosting
AP	Average precision
AUROC	Area under the receiver operating characteristic curve
FPR	False-positive rate
TP / FP	True positive / false positive
TN / FN	True negative / false negative
RPC	Remote procedure call
SOL	Native currency unit of Solana
SGD	Stochastic gradient descent
API	Application programming interface
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
SQL	Structured Query Language
CI	Confidence interval

CHAPTER 1

INTRODUCTION

1.1 Background

HOMIE uses earlier Solana records to rank transactions by execution-failure risk. Transactions may transfer SOL or interact with blockchain programs.

A wallet may have no usable history in the collected sample. However, the accounts involved in its transaction may have appeared in other transactions. HOMIE uses these shared histories as additional input to a machine learning model. Here, an account is a blockchain data address; it does not necessarily represent another person's wallet.

1.1.1 Wallets, signers and accounts

A wallet helps a user manage signing keys and approve transactions. The signer is the address whose signature authorises a transaction. The primary signer used in this study is the first signer in the sampled record. A transaction may also reference accounts holding balances, program state or other data. These referenced accounts are not necessarily people or wallets [1, 2].

The distinction matters because the proposed model uses the history of *participation* in transactions. If an account repeatedly appears in failed operations, that pattern may be informative about a later operation involving it. The model does not infer that the account owner caused those failures or acted maliciously.

1.1.2 Writable and read-only account roles

A writable account is declared as an account whose state may be changed by the transaction. A read-only account may be read without modification. These declarations describe access permissions, not a guarantee that an account will actually change. The project separates the primary signer from other writable accounts to avoid treating the signer's own history as new contextual evidence.

Read-only templates in this project are sets of declared read-only account addresses. They can capture recurring transaction contexts, but are not verified lists of programs invoked by each transaction. This definition follows the fields available in the selected archive.

1.1.3 Failure, rejection and unknown outcome

An execution failure is a recorded unsuccessful execution outcome. A candidate may instead be rejected before submission, or an attempted transaction may never appear in the observed archive. These cases must not be merged into the same label. The historical model is evaluated on included transactions with recorded outcomes. The application separately records simulation results and receipt availability.

1.2 Problem Statement

Signer-history models have little direct evidence when a signer has no available outcomes. The research question is: *Does non-signer writable-account history improve failure ranking beyond transaction structure, signer history, read-only context and a learned identity control?*

1.3 Motivation / Need for the Project

A model that depends mainly on signer history can perform well on repeat participants while providing little information for a signer without earlier outcomes. Reporting only the aggregate score can hide this weakness. HOMIE therefore measures performance on a defined missing-history subgroup as well as examining the broader dataset.

Two subgroups are distinguished. A cold-history transaction has no matured primary-signer outcome available at its checkpoint. A first-appearance transaction is the first observed primary-signer occurrence in the sampled replay. A signer can appear repeatedly within the one-hour outcome delay and still remain cold. First-appearance evaluation reduces this particular source of repetition.

1.4 Objectives

1. Construct reproducible features from public transaction records using only earlier available outcomes.
2. Compare a baseline and a writable-context model on chronological test periods.
3. Evaluate first-appearance transactions separately from aggregate performance.
4. Implement a usable interface for inference, transfer checks and traceable outcome recording.
5. Verify the fee mechanism using an isolated local validator.

1.5 Scope of the Project

HOMIE retains the name of the original DeFi-companion concept, but its implemented focus is transaction-risk assessment. The work contributes an empirical evaluation of account-context features and an integrated prototype. Histogram gradient boosting is an established

algorithm; the research contribution is the feature comparison and evaluation setting, rather than a newly invented classifier.

The predicted label is execution failure, not fraud, malicious intent or investment risk. "Cold history" means unavailable signer outcomes in the sampled replay; it does not establish that a wallet is newly created on Solana.

1.6 Significance and Expected Contribution

Consider a signer with no available history that submits an operation referencing accounts A and B. Earlier transactions from other signers have involved these accounts. The baseline sees transaction structure and read-only context. The extended model additionally receives summaries of earlier outcomes involving writable A and B. It may assign a different score because these summaries supply evidence that the signer-only history lacks.

This example explains the mechanism; it is not an additional measured result. The model can increase or decrease a score depending on the learned pattern. A lower score alone does not prove an improvement. The recorded outcome and performance across the test set determine whether the predictions are useful.

1.7 Organization of the Report

Chapter 2 reviews relevant literature. Chapter 3 describes the proposed system, requirements and architecture. Chapter 4 explains the research methodology. Chapter 5 presents implementation and experimental verification. Chapter 6 reports results and analysis. Chapter 7 concludes the work and identifies future scope. References follow the concluding chapter.

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction

This chapter reviews Solana execution studies, relational learning and the methodological foundations used in HOMIE.

2.2 Review of Existing Work

Zheng et al. study failed Solana transactions through large-scale measurements of initiators, programs and error patterns [3]. Their work establishes transaction failures as an empirical research subject. HOMIE focuses more narrowly on prediction when signer history is unavailable and on testing explicit shared-account history.

Solana's official documentation describes transaction fees and the RPC simulation interface [4, 5, 6]. Simulation executes a candidate against a particular state; a statistical history model estimates patterns from past observations. These are different sources of information and require separate evaluation.

The scikit-learn histogram gradient boosting classifier provides the implemented tabular learning method [7]. Public SolArchive records supply the historical data [8].

Table 2.1: Relationship between existing resources and HOMIE

Resource	Relevant contribution	HOMIE focus
Solana failure study	Characterisation of failed transactions	Prediction under missing signer history
Solana RPC simulation	Execution check against current state	Displayed alongside independent ML scores
Histogram gradient boosting	Established tabular classifier	Controlled account-history feature comparison
SolArchive	Public transaction archive	Chronological sampled evaluation

2.2.1 Temporal Relational Learning

BRIGHT, proposed by Lu et al., addresses real-time fraud detection using graph neural networks [9]. It explicitly prevents future transactions from sending information into predictions about earlier transactions and separates historical computation from real-time inference. The relevant lesson for HOMIE is the importance of an information boundary when relationships between entities are used as predictors.

HOMIE uses tabular participation summaries rather than a graph neural network. Its target is Solana execution failure rather than e-commerce fraud. BRIGHT is therefore related methodological work, not a directly evaluated baseline. No claim of outperforming BRIGHT is made because the datasets, labels and operating conditions differ.

2.2.2 Boosting, Calibration and Evaluation

Friedman formulates gradient boosting as stage-wise function approximation [10]. This provides the methodological foundation for building a predictor through successive corrections. HOMIE uses an existing histogram boosting implementation, so the algorithm is credited as established prior work rather than presented as the project's invention.

XGBoost by Chen and Guestrin describes a scalable tree-boosting system [11]. LightGBM by Ke et al. develops efficient gradient boosted decision-tree training [12]. These are relevant alternatives in the tree-ensemble literature. The current study uses scikit-learn's histogram classifier; XGBoost and LightGBM results are not claimed as implemented comparisons. Their inclusion identifies the wider algorithm family and potential future benchmark alternatives.

Niculescu-Mizil and Caruana examine predicted probabilities and calibration across supervised learners [13]. Their distinction between a discriminative score and a reliable probability motivates reporting calibration alongside ranking. HOMIE's isotonic follow-up is evaluated on chronological validation data and does not erase the need to inspect performance on later dates.

Saito and Rehmsmeier discuss precision–recall plots for imbalanced binary classification [14]. The relevance is direct: failures form a minority of the reserved first-appearance group, and an always-success classifier would have high accuracy while missing every failure. Reporting AP, recall and false positives makes this trade-off visible.

Kapoor and Narayanan discuss leakage and reproducibility problems in ML-based science [15]. In HOMIE, chronology, delayed labels and exclusion of current execution outputs are safeguards against this class of error. These safeguards define what information the model can use; they do not replace external validation.

Table 2.2: Methodological literature and its use in the project

Work	Relevant concept	Use in HOMIE
Friedman [10]	Gradient boosting	Foundation of the fitted classifier
Chen and Guestrin [11]; Ke et al. [12]	Efficient tree ensembles	Related methods, not claimed experiments
Weinberger et al. [16]	Feature hashing	Fixed-dimensional identity representation
Niculescu-Mizil and Caruana [13]	Probability calibration	Separating ranking from numerical reliability
Saito and Rehmsmeier [14]	Imbalanced evaluation	Precision–recall interpretation
Kapoor and Narayanan [15]	Leakage and reproducibility	Information-availability safeguards

2.3 Comparative Analysis

2.3.1 Signer-based prediction

Signer history can summarise repeated behaviour efficiently. Its weakness in this study is absence of evidence for some signers at prediction time. Replacing missing history with a global average supplies a reasonable default but does not distinguish otherwise similar cold-history transactions. Shared-account features are tested as a way to add information in that setting.

2.3.2 Identity-based learning

A model may learn that particular addresses recur with particular outcomes. This can explain apparent benefits from contextual features. The identity-control experiment gives both compared models a score learned from role-tagged account identities, including writable identities. The remaining comparison asks whether explicit historical summaries help beyond that learned score.

2.3.3 Execution simulation

Simulation evaluates a candidate transaction against a network state. It can identify a concrete error without needing a statistical training set. Historical ML could be complementary if it captured useful residual information, but demonstrating that requires observations beyond a retrospective archive. HOMIE includes both mechanisms in the application while evaluating their findings separately.

2.4 Research and Design Gap

The tested hypothesis is that explicit writable-account participation summaries provide incremental ranking signal. A role-tagged identity learner is supplied to both compared models to make simple identity memorisation a stronger baseline. The positive results support this bounded empirical contribution; they do not establish universal novelty across all blockchain prediction research.

2.5 Summary

The report addresses three questions. First, does writable context improve failure ranking for sampled cold-history transactions? Second, does the improvement remain for first appearances when both models receive an identity score? Third, what happens when historical scores are turned into practical warning policies?

Chronological experiments address the first two questions. Calibration and a cost audit address the third, with fee behaviour verified separately on a local validator.

CHAPTER 3

PROPOSED SYSTEM AND SCOPE

3.1 Problem Formulation

Signer-history models have little direct evidence when a signer has no available outcomes. The research question is: *Does non-signer writable-account history improve failure ranking beyond transaction structure, signer history, read-only context and a learned identity control?*

3.2 Proposed Solution

HOMIE combines a historical ML comparison with a transfer-checking application. The extended model adds writable-account participation history to the baseline. Network checks and recorded outcomes are displayed separately from model scores.

3.3 Requirements

3.3.1 Functional and Non-Functional Requirements

The system must support historical replay, fitted-model inference, public-history refresh, native SOL transfer checks and outcome recording. It must report whether data is fresh enough for live scoring. It must preserve the exact candidate message and model provenance before associating a later receipt with a decision.

Insufficient or stale live history must withhold an ML score. Missing network responses must remain incomplete checks. Unknown receipt outcomes must remain unknown. The web application performs checks without signing or broadcasting transactions.

The live flow accepts the selected network, sender address, recipient address and transfer amount. Addresses must be valid public keys. Amounts are parsed exactly into lamports; arbitrary floating-point rounding must not alter the intended transfer. The supported candidate is a plain native SOL transfer. Arbitrary token swaps and multi-instruction DeFi transactions are outside this implemented checker.

Outputs include a clear current-network finding, the quoted fee, the amount-plus-fee requirement and any balance shortfall. When valid history and model artifacts are available, the response also contains actual model scores and provenance. It records whether simulation occurred and whether signature verification was enabled. Each saved observation has an identifier for later lookup.

3.3.2 Hardware and Operating Requirements

Historical experiments require local storage for selected archive data and sufficient memory for feature tables. Live checks require an Internet connection and an available RPC endpoint. A browser and the Python environment run the application; local fee verification additionally requires a Solana validator installation. Minimum hardware throughput has not been benchmarked, so no unsupported minimum performance guarantee is specified.

3.3.3 Software Requirements

Table 3.1: Implemented technology stack

Component	Technology and purpose
Data processing	Python, pandas [17], NumPy [18] and PyArrow for archive extraction and features
Machine learning	scikit-learn [19] histogram gradient boosting; SGD logistic identity control; isotonic calibration follow-up
Backend	FastAPI served with Uvicorn
Interface	Browser-based HTML, CSS and JavaScript
Study storage	SQLite for immutable decisions and separately attached receipts
Blockchain access	Solana JSON-RPC for blocks, balances, fee quotes and simulation
Verification	pytest and an isolated Solana local validator

3.4 System Architecture / Block Diagram

The architecture separates historical experimentation from live transfer analysis. Both routes reuse participation-history features, but live scores are experimental because current sampling and network conditions differ from the historical training data.

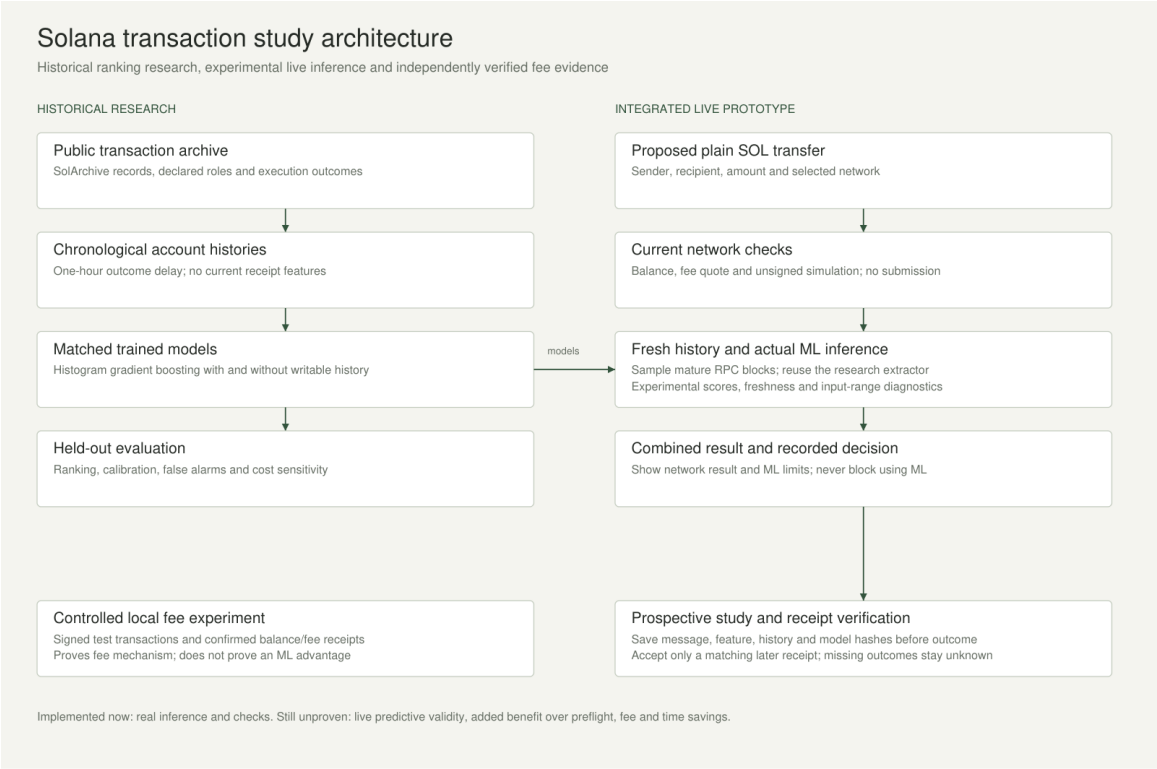


Figure 3.1: Implemented research and application architecture.

3.4.1 Transaction-Check Workflow

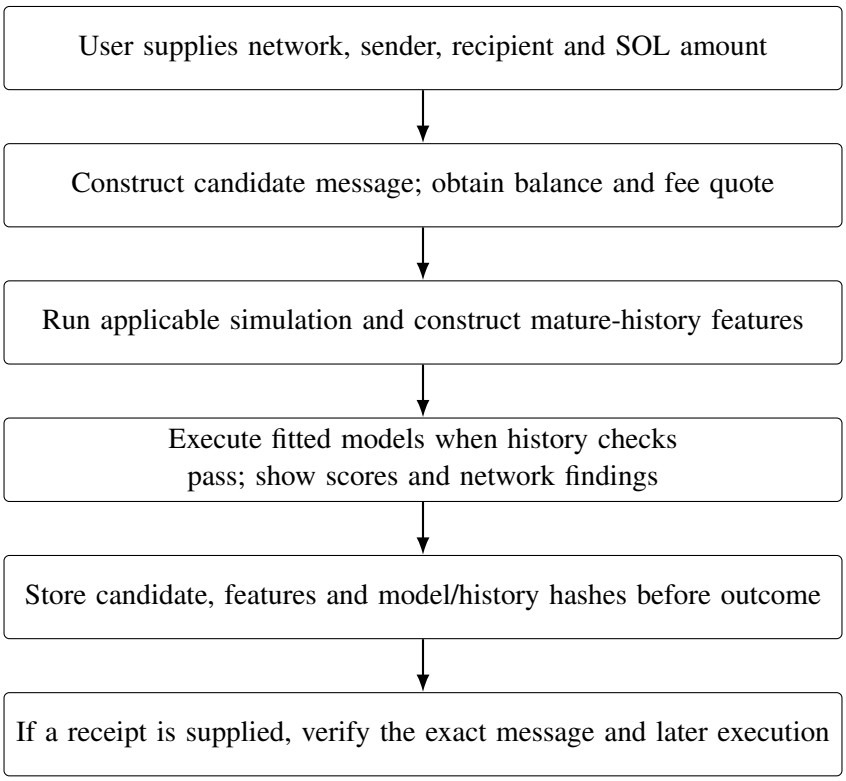


Figure 3.2: Live checking and outcome-recording workflow.

3.4.2 Data and Responsibility Boundaries

The archive provides transaction outcomes for retrospective evaluation. The RPC service supplies present network observations. The ML component consumes account-role and historical features; transfer amount is handled by the balance and simulation checks. The study store preserves the decision before the receipt is attached. This separation makes it possible to identify which component produced each finding.

3.5 Functional / System Components

Table 3.2: Implemented modules and responsibilities

Module	Input	Responsibility
Archive extraction	Selected date and shard	Retrieve selected columns and record provenance
Participation history	Chronological account roles and matured labels	Produce structural and contextual features
Model training	Training and validation feature tables	Fit controlled classifiers and retain artifacts
Live history	Public RPC blocks	Prepare network-specific mature snapshots
Transfer checker	Addresses, amount and network	Construct message; check balance, fee and simulation
Study store	Decision payload and later signature	Preserve observation; verify matching receipt
Web interface	User selection or transfer input	Display results and inspection links

Table 3.3: Users and supported activities

User	Activity	System output
Researcher	Compare baseline and context predictions	Metric tables, saved predictions and provenance
Project evaluator	Inspect a historical example	Features, model scores and revealed recorded result
Transfer user	Check a proposed native SOL transfer	Balance, fee quote, simulation finding and experimental score
Study operator	Link a later transaction receipt	Verified matching outcome or explicit rejection

3.6 Scope and Constraints

The historical workflow records extraction settings and derived-file hashes. Conflicting records or date mismatches must not be silently repaired into convenient data. Training, validation and test boundaries are defined chronologically. Feature columns and model

settings are retained so exported inference consumes the same type and order of inputs used during fitting.

The live collector and model scorer must agree on the network. A mainnet-trained experiment may be inspected on devnet for functional testing, but devnet output is not evidence of equivalent predictive performance. A snapshot's collection timestamp and the maturity of its rows are checked separately.

The interface provides concise findings with inspectable evidence. Separate modules support collection, inference, transfer checks and receipt storage.

Invalid inputs are rejected before RPC calls. Unavailable RPC or history data produces an explicit incomplete result. Receipts must match the checked message; a changed blockhash requires a new observation.

CHAPTER 4

METHODOLOGY

4.1 Objective of the Study

The study tests whether writable-account history adds failure-ranking information when signer history is unavailable, including a stronger comparison with a shared identity score.

4.2 Method / Approach

The historical source is SolArchive, licensed CC BY 4.0 [8]. The pinned dataset revision is:

`6588cad2832e62be03972ac80d69c6af11d631c8`

One selected Parquet row group per date is used. The initial four windows contain 1,588,408 raw records before duplicate removal and exclusions. This is a sparse sample, not complete chain coverage.

Selected fields include transaction signature, account addresses and roles, block timestamp, block slot and execution status. The target is binary execution failure. Original instruction payloads, unlanded attempts and paid fee amounts are not available in the selected historical table.

4.3 Measurement / Evaluation Parameters

Table 4.1: Chronological experimental windows (2025)

Window	Training	Validation	Testing
Initial	Nov. 1	Nov. 2	Nov. 3
Confirmation	Nov. 8	Nov. 9	Nov. 10
Extension	Nov. 22	Nov. 23	Nov. 24
Reserved refined	Nov. 29	Nov. 30	Dec. 1
Further test	Dec. 3	Dec. 4	Dec. 5

Average precision (AP) measures how well failures rank ahead of successful transactions across score cut-offs. It is not classification accuracy. AUROC, Brier loss, recall and false-positive rate provide complementary information. Paired bootstrap resampling of hourly blocks uses 500 draws to estimate uncertainty in AP differences.

4.3.1 Metrics and interpretation

Let TP denote flagged failures, FP flagged successes, FN missed failures and TN unflagged successes. For a fixed warning threshold,

$$\text{Precision} = \frac{TP}{TP+FP}, \qquad \text{Recall} = \frac{TP}{TP+FN}, \tag{4.1}$$

$$\text{FPR} = \frac{FP}{FP+TN}, \qquad \text{Accuracy} = \frac{TP+TN}{TP+FP+FN+TN}. \tag{4.2}$$

A classifier that always predicts success has accuracy equal to the success prevalence. On the reserved first-appearance group with 5.88% failures, that simple rule would have about 94.12% accuracy but identify no failures. This is why accuracy alone is unsuitable for judging the research question. Precision–recall analysis is particularly informative for imbalanced binary tasks [14].

Average precision summarises the precision–recall curve as $\sum_k (R_k - R_{k-1})P_k$, where P_k and R_k are precision and recall as the score threshold changes [20]. AUROC describes ranking across positive and negative cases. Brier loss, $N^{-1}\sum_i (s_i - y_i)^2$, assesses the numerical predictions against binary outcomes; lower values are better [21].

4.3.2 Uncertainty and experimental discipline

The paired-hour bootstrap samples hourly blocks and computes the difference between both models on the same resampled observations. Pairing preserves their direct comparison. Grouping by hour accounts for some temporal clustering rather than treating every transaction as independent. The resulting interval is conditional on the fitted models and sampled periods; it does not measure all possible deployment uncertainty.

The reserved refined endpoint was specified after earlier exploratory diagnostics and before inspecting the fourth window's outcomes. It is a reserved historical test, not a publicly preregistered live trial. The further December window supplies another chronological comparison. This distinction makes the development of the research question clear without presenting exploratory choices as fixed from the start.

4.4 Tools, Equipment and Resources

Python, pandas, NumPy and PyArrow prepare chronological tables. Scikit-learn fits classifiers and computes metrics. Public SolArchive data supports the historical study; a Solana local validator supports the separate fee experiment.

4.5 Experimental Procedure

The research path transforms archive records into a feature table, then into fitted models and held-out predictions. Test labels are used to evaluate those predictions and to update

later history only after the defined delay. They are not used to refit the gradient boosting model during the test day.

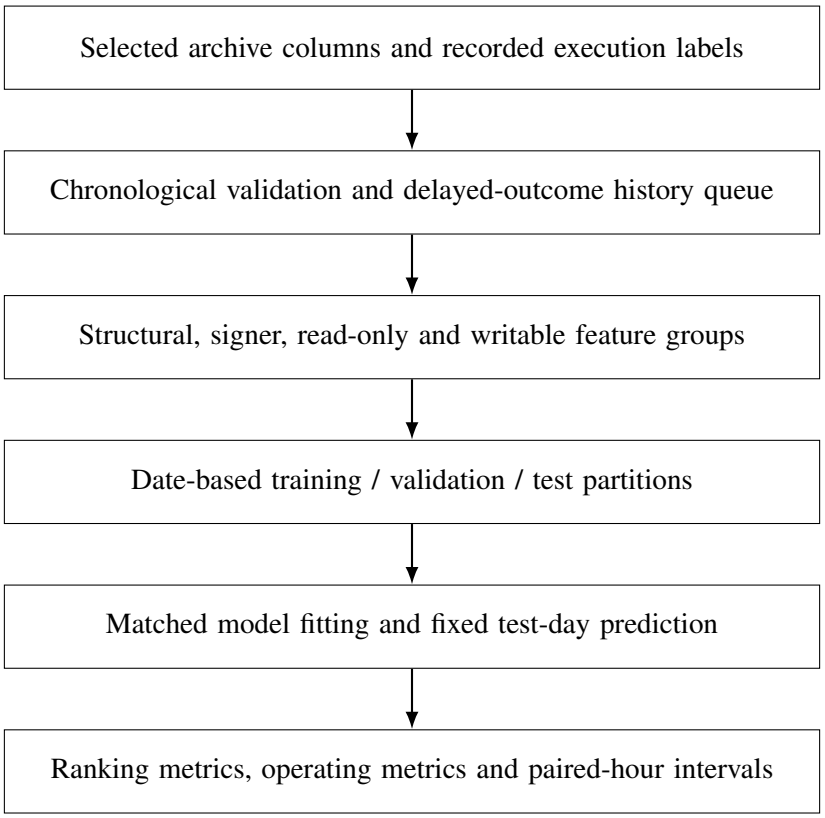


Figure 4.1: Historical research data flow.

4.6 Model Architecture

Histogram gradient boosting combines successive decision trees to reduce classification loss, following the gradient boosting framework [10]. Numeric feature values are grouped into bins for efficient split selection [7]. Both primary models use the same classifier settings so the comparison concerns the added features.

Table 4.2: Histogram gradient boosting configuration

Parameter	Value
Maximum iterations	150
Maximum leaf nodes	15
Minimum samples per leaf	40
L2 regularisation	5
Early stopping	Disabled
Seeds averaged	42, 73, 101

The fits are effectively deterministic; the three seeds are not independent experimental replications. The baseline includes structure, signer history and read-only/template history. The context model adds non-signer writable-account history.

4.6.1 How boosting produces a score

The classifier learns a sequence of trees. Each new tree contributes a correction to the model's current decision function. In binary classification, the decision function $z(x)$ is mapped to a score through a logistic link:

$$z_M(x) = z_0 + \eta \sum_{m=1}^{M} h_m(x), \qquad s(x) = \frac{1}{1 + e^{-z_M(x)}}. \tag{4.3}$$

Here h_m is a fitted tree and η is the learning-rate parameter. Training reduces binary log loss, which penalises confident incorrect predictions. A probability-shaped output from this function still requires calibration assessment before being interpreted as an accurate event frequency in a new population.

4.6.2 Why a tabular model fits this implementation

The input is a fixed-length numeric representation of transaction structure and account history. A tree ensemble can learn non-linear combinations, such as different effects of a historical rate at low and high observation counts. It does not require a sequence of natural-language queries or a language model. The implemented method also allows a controlled feature ablation with the same learner on both sides.

Deep learning and graph models are possible extensions, but their inclusion would require a defined representation, training procedure and fair comparison. They are not necessary to describe the actual implemented contribution. The report therefore identifies histogram gradient boosting as the main algorithm and distinguishes the logistic identity control from it.

4.6.3 Identity Control and Calibration

A stronger comparison uses an SGD logistic learner [22] with role-tagged identities and read-only templates hashed into 2^{20} dimensions. Feature hashing provides a fixed-dimensional representation of large categorical spaces [16]. Binary features are L2 normalised; regularisation is 0.0001. Its chronological predictions are supplied to both gradient boosting models. Labels mature after one hour, with ten-minute learning batches introducing a small additional update delay.

A separate operational follow-up fits isotonic calibration [23, 13] on early cold-history validation data and chooses thresholds on later validation data. This follow-up is distinct from the identity-controlled primary comparison. Live application outputs remain experimental raw scores rather than validated live failure probabilities.

4.7 Error Sources and Mitigation

Temporal leakage, repeated signers and identity memorisation can distort an apparent improvement. Delayed labels, a first-appearance endpoint and a shared identity control address these risks. Sparse sampling and changing network conditions remain limits of the evaluation.

4.8 Data Processing and Analysis

At transaction time t, an earlier outcome contributes only if its inclusion timestamp is at least one hour earlier. Features are computed before the current outcome is scheduled for later availability. The first six training hours build history. Rows whose labels would cross the next date boundary are purged from training and validation.

For account a in role r, the smoothed historical failure rate is

$$q_r(a,t) = \frac{f_r(a,t) + 20b(t)}{n_r(a,t) + 20}, \qquad b(t) = \frac{F(t)+1}{N(t)+2}. \tag{4.4}$$

Here f_r and n_r are available failures and observations for that role; F and N are their global counterparts. Features summarise seen fractions, log counts and failure-rate mean, maximum, minimum and standard deviation. Ten common infrastructure addresses are excluded from contextual histories and represented through structural indicators.

4.8.1 Worked history example

Suppose an account has participated in ten matured observations, including two failures, and the global available failure rate is 0.05. With smoothing strength 20, its rate is

$$q = \frac{2 + 20(0.05)}{10 + 20} = 0.10. \tag{4.5}$$

With no account observations, the same rule returns the global rate. This avoids assigning an extreme rate to an account solely because one observed transaction failed. The example uses illustrative counts to explain the implemented formula; it is not a result sampled from the evaluation.

4.8.2 Feature groups

Table 4.3: Feature groups and their interpretation

Group	Examples	What it describes
Structure	Account, signer and writable counts; duplicates	Declared transaction composition
Signer history	Smoothed failure rate and log count	Available outcomes for the primary signer
Read-only history	Seen fraction, counts and rate summaries	Prior participation of read-only accounts
Read-only template	Template rate and count	Recurring declared account set
Writable history	Seen fraction and rate/count summaries	Prior participation of non-signer writable accounts
Infrastructure	Presence indicators	Common infrastructure accounts without pooled context histories

Seen fraction is a coverage measure: a value of 14.3% means that proportion of the relevant accounts has earlier available observations. It is not an accuracy score or a failure probability. The failure rates and coverage variables give the model both a pattern and information about how much history supports it.

4.8.3 Leakage prevention

Using the current transaction's error message would make failure prediction trivial after execution. Leakage of unavailable information can invalidate an ML evaluation [15]. HOMIE therefore excludes current logs, errors, consumed compute, balances, paid fees and block position from its historical model inputs. Time controls chronology and information availability but is not supplied as a numeric predictor.

Consider a transaction included at 10:20. Under the study's one-hour delay, its label cannot contribute to a 10:45 prediction. It becomes eligible at 11:20. The delay is an explicit research assumption. Inclusion timestamps do not reconstruct the exact information available at historical submission time.

4.8.4 Replay procedure

For each chronological record, the feature builder first releases eligible earlier labels from the pending queue. It then resolves the current account roles and computes features from that available history. Only after the feature row is produced is the current outcome scheduled for release one hour later. This order is reused across the research and live-history extraction paths.

CHAPTER 5

IMPLEMENTATION AND EXPERIMENTAL WORK

5.1 Implementation Environment

The application uses FastAPI and Uvicorn, browser-based HTML, CSS and JavaScript, and SQLite for observation storage. Research outputs and fitted model artifacts are loaded from the project workspace.

5.2 Module-wise Implementation

The FastAPI application [24] provides historical replay at `/`, fitted inference at `/predict`, transfer checks at `/check`, integrated analysis at `/live` and fee verification at `/fee-proof`. The backend reconstructs plain SOL transfer messages, handles exact lamport amounts and queries Solana RPC. It obtains a recent blockhash [25], requests a fee quote for the constructed message [26] and reads the sender balance [27].

Live snapshots must meet freshness and history requirements: at least 100 records spanning six hours, with a snapshot no older than one hour. Public blocks are retrieved using the documented block RPC [28]. Only outcomes aged at least one hour enter history. The same extractor used in research builds live features. Scores are withheld when checks fail.

SQLite records candidate-message, feature, model and history hashes. Database triggers protect decisions and receipts from mutation. Receipt verification checks the exact candidate message, signature, later slot and timestamp, and consistency of fee and balance changes. The receipt is retrieved through the transaction RPC [29]. An unrelated receipt cannot be used as evidence for a decision.

5.2.1 Balance, fee and amount calculation

For balance B, requested amount A and quoted fee F, the checker calculates

$$\text{required} = A + F, \qquad \text{shortfall} = \max(0, A + F - B). \tag{5.1}$$

For example, if a recorded check has a balance of 0.5120009 SOL, a requested transfer of 1 SOL and a quoted fee of 0.000005 SOL, the shortfall is 0.4880041 SOL. This corresponds to the saved insufficient-balance verification case. A sufficient balance passes this arithmetic check but may still encounter a separate simulation error.

5.2.2 Simulation modes

The standard web check creates an unsigned candidate and calls simulation with signature verification disabled. This allows a user to inspect supported execution conditions without supplying a private key. The result concerns the candidate at the checked state, not whether a valid signature will later be provided.

A separate signed-observation route accepts an already signed plain transfer, preserves its blockhash and enables signature verification during simulation. It still does not submit the transaction. The two modes are labelled separately because an unsigned check and signed simulation do not establish the same facts.

5.2.3 Model provenance and range checks

The stored response includes hashes identifying the model artifact, history snapshot and feature values. These support later inspection of which inputs produced the score. Live input values are compared with central training ranges as a diagnostic. A value within those ranges is not proof that a new transaction has the same outcome distribution as the training set.

Changing the amount may change the balance or simulation finding while leaving the ML score unchanged, because the current ML features do not encode the amount. This behaviour follows the feature definition and is useful to explain during a demonstration. The model assesses account context; the network checker assesses the constructed amount-specific candidate.

5.2.4 Decision Storage

The implemented SQLite store uses a decision table, a receipt table and an abstention table. SQLite triggers support rejecting updates and deletions of protected records [30]. A decision has a generated identifier, serialised payload, payload hash and recording timestamp. The payload contains the candidate and the relevant check and model observations. Receipts are stored separately so a later outcome cannot overwrite what was known at prediction time.

Table 5.1: Study-store entities

Entity	Main fields	Relationship or constraint
Decisions	id, payload, payload hash, recorded time	Primary record; update and delete triggers prevent mutation
Receipts	decision ID, signature, payload, recorded time	At most one receipt per decision; unique signature; immutable record
Abstentions	decision ID, reason, recorded time	Optional record associated with a decision

An abstention is a recorded decision not to proceed; by itself it cannot establish what would have happened. Similarly, the lack of an attached receipt does not establish success, failure or avoided fees. Application-level immutability and hashes improve traceability, but do not make a local database immune to a privileged administrator replacing its files.

5.2.5 Receipt Verification

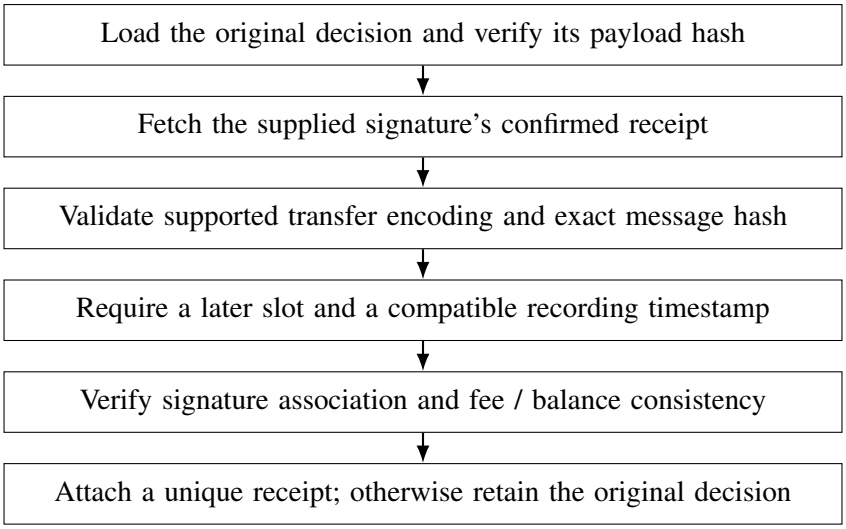


Figure 5.1: Checks before a receipt becomes an observed outcome.

5.3 Experimental Setup and Test Cases

Table 5.2: Test inputs and expected safeguards

ID	Scenario	Expected response
T01	Current label supplied to replay	Current prediction excludes that outcome
T02	History timestamp is in the future	Reject invalid history
T03	Snapshot older than one hour	Withhold ML score and request refresh
T04	Mainnet request with devnet history	Do not score using the wrong network
T05	Amount exceeds balance plus fee budget	Return the exact shortfall
T06	Simulation reports rent-related error	Return an actionable current-state finding
T07	Receipt belongs to another message	Reject outcome attachment
T08	Receipt predates recorded observation	Reject retrospective attachment
T09	No confirmed receipt is returned	Keep outcome and paid fee unknown
T10	Attempt to overwrite stored decision	Database trigger rejects mutation
T11	Unsigned input sent to signed observer	Reject unsupported signed observation
T12	Repeated history-refresh request	Reuse the running job rather than duplicate it

These scenarios are grounded in the implemented tests and recorded checks. Network-dependent examples are observations from their recorded run; their outputs can change as balances and chain state change. Unit tests exercise controlled inputs so that key invariants can be repeated independently of public RPC availability.

5.3.1 Local Experiment Procedure

The verification script starts its own loopback validator and creates throwaway test accounts funded through local airdrops. It constructs an invalid transfer whose execution violates an account minimum-balance requirement. The checker observes the error. The experiment then compares a stopped candidate, ordinary preflight rejection and a deliberately forced execution failure with preflight disabled only inside the isolated test.

For the executed case, the script retrieves the confirmed receipt and compares the reported fee with the payer's before-and-after balance. The failed transfer does not deliver the requested principal, but the execution fee is observed. A valid transfer separately verifies that the sender's balance decreases by principal plus fee. The script stops the validator instance it started.

5.4 Prototype / System Implementation

The recorded integration checks include a small transfer, insufficient balance, a rent-related failure and a read-only mainnet case. The system returns current findings and, where historical inputs qualify, actual fitted-model inference. HTTP success alone is insufficient: the response content and the absence of a broadcast path are also inspected.

The presentation connects inputs, findings and the stored decision, supported by historical results and local receipts.

5.5 Testing and Validation

Targeted automated tests cover feature timing, inference, transfer-message construction, policy arithmetic and receipt integrity. The recorded verification run reports 45 passing targeted tests. This count refers to the selected project checks, not a claim about every test in the repository.

Table 5.3: Implemented checks and recorded verification

Area	Expected behaviour	Recorded verification
History maturity	Exclude current and immature outcomes	Targeted tests passed
Freshness and network	Withhold scores for stale or mismatched data	Targeted tests passed
Insufficient balance	Show amount-plus-fee shortfall	Live HTTP case verified
Rent constraint	Surface simulation error	Live HTTP case verified
Valid small transfer	Show simulation pass at checked state	Live HTTP case verified
Read-only operation	Do not broadcast from web checks	Targeted tests and HTTP checks
Receipt integrity	Reject unrelated, old or duplicate receipts	Targeted tests passed
Local receipt flow	Attach matching later receipt to stored decision	Local validator verification

5.5.1 Fee-Mechanism Experiment

An isolated local validator and throwaway keys are used to compare a deliberately invalid transfer under different submission conditions. Normal preflight rejects the invalid candidate without a paid transaction fee. In a test-only path, disabling preflight allows a landed execution failure; the receipt and balance change record 5,000 lamports. A successful transfer separately checks principal-plus-fee accounting.

This experiment verifies fee behaviour and traceable outcomes. It does not establish that the ML model outperforms ordinary preflight. The local validator uses test funds and the production web flow does not bypass preflight or submit transactions.

CHAPTER 6

RESULTS, ANALYSIS AND DISCUSSION

6.1 Results

Table 6.1: Cold-history AP without the identity-score control

Test date	Transactions	Baseline AP	Context AP
Nov. 3	19,621	0.3193	0.3787
Nov. 10	20,636	0.2249	0.2514
Nov. 24	21,028	0.2052	0.2231
Dec. 1	19,577	0.1734	0.2185

All four point estimates favour writable context, but the November 24 interval for the difference crosses zero. Results therefore do not imply an advantage in every period.

Table 6.2: First-appearance results with the identity-score control

Test	Rows	Baseline AP	Context AP	AP gain, 95% interval
Dec. 1	17,904	0.1688	0.2004	0.0316 [0.0168, 0.0501]
Dec. 5	19,515	0.2207	0.2393	0.0186 [0.0095, 0.0288]

The reserved December 1 endpoint has 5.88% failure prevalence. Its AUROC rises from 0.7288 to 0.7545. The further December 5 result also supports additional ranking signal beyond the shared identity control.

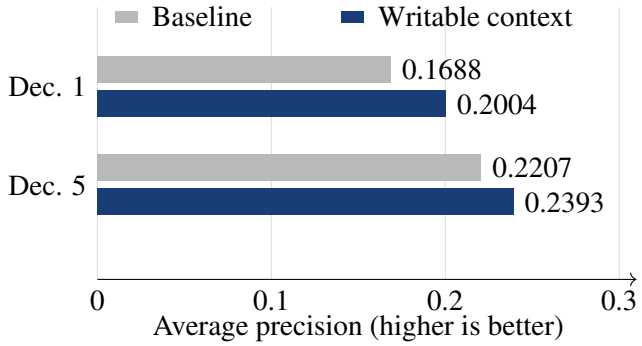


Figure 6.1: Identity-controlled first-appearance AP on two test dates. The plot shows ranking performance, not classification accuracy.

6.1.1 Fee-Policy and Local Results

Table 6.3: Measured local fee-mechanism outcomes

Local condition	Paid fee (lamports)
Check candidate and stop	0
Normal preflight rejects invalid candidate	0
Force landed execution failure in isolated test	5,000

A retrospective policy audit covers 102,443 cold-history transactions with 6,410 failures. At an assumed false-warning cost five times a failed-attempt fee, context warnings cover 107 failures and 60 successes while missing 6,303 failures. Hypothetical utility is $107 - 5(60) = -193$ fee units. These units are assumed costs, not measured SOL savings.

6.2 Analysis

On December 1, the AP difference is $0.2004 - 0.1688 = 0.0316$, or about 3.16 AP percentage points. Relative to the baseline AP, this is approximately an 18.7% increase. This relative change is a ranking comparison; it must not be described as 18.7% fewer failed transactions or an 18.7% reduction in fees.

On December 5, the corresponding difference is approximately 0.0186. Both first-appearance intervals exclude zero under the specified paired-hour analysis. The second test has a smaller gain, which is consistent with the usefulness of context changing across sampled dates. Neither result implies correct predictions for every individual transaction.

On December 1, inherited validation thresholds yield false-positive rates of 22.66% for the baseline and 28.65% for the context model. Recall rises from 50.90% to 64.48%, but this comparison does not hold false positives constant. Brier loss worsens from 0.0967 to 0.1218. Better ranking therefore does not automatically mean better calibrated probabilities or a better warning policy.

In the interface, a displayed score such as 30.8% is a model output for one transaction, not 30.8% model accuracy. Whether a case is flagged depends on a threshold; judging a policy requires outcomes across many transactions and attention to both missed failures and false alarms.

6.3 Discussion

The interface displays the fraction of relevant accounts with available history so the user can see the amount of contextual coverage. A high coverage fraction means the model has earlier observations for more of those accounts. It does not imply high-quality or representative observations, and it does not guarantee confidence in the prediction.

Two models may give different scores to the same transaction because one has additional features. For the earlier example with scores 41.5% and 30.8%, a threshold can flag one and not the other. If the transaction succeeds, the lower score may be more consistent with that single outcome; if it fails, the higher score may be. Aggregate ranking, calibration and operating metrics are needed to judge which approach is better overall.

6.4 Comparison with Existing Approaches

The baseline and writable-context models use matched classifier settings and chronological periods. The identity-controlled comparison supplies the same learned score to both. These within-dataset comparisons isolate the added feature group. Literature methods using different targets and datasets are not claimed as directly outperformed.

6.5 Significance and Implications

The project establishes that the fitted models can be executed on constructed features, current network checks can be performed for supported transfers, and an observation can be preserved before a matching receipt is attached. The fee experiment establishes that the stage at which a transaction fails matters for fee accounting. These are separate, inspectable engineering outcomes.

The policy audit also provides an informative negative finding: the tested ranking gains do not translate automatically into favourable pooled cost utility. A useful final-year research report includes that result because it identifies the actual deployment problem. Calibrating a score, choosing a warning threshold and evaluating the cost of delaying a successful transaction are distinct from fitting the classifier.

6.6 Potential Applications

The prototype can support research inspection, prioritisation of transactions for review, and diagnosis of supported native SOL transfer problems. Balance and simulation findings can guide a user before submission. Automatic transaction blocking or economic savings require separate policy evaluation.

CHAPTER 7

CONCLUSION AND FUTURE SCOPE

7.1 Summary of Work

HOMIE implements an end-to-end transaction-risk research system on Solana. Public records are transformed into time-respecting account features, established machine learning models are compared, and predictions are exposed through an application with live checks and traceable outcomes.

The strongest finding is that writable-account participation history adds failure-ranking signal for sampled first-appearance signers, including comparisons with a learned identity control. The project also verifies the fee mechanism and the decision-to-receipt workflow. Its results distinguish predictive ranking from probability calibration and economic benefit.

7.2 Conclusions

Writable-account history improves the reported first-appearance AP on two chronological tests with a shared identity control. The warning-policy audit shows that ranking improvement alone does not guarantee favourable cost utility.

7.3 Contributions

The project integrates chronological participation features, controlled model comparisons, a working inference interface and a decision-to-receipt workflow. A separate local experiment verifies execution-fee behaviour.

7.4 Utility / Applications

HOMIE provides an inspectable system for studying transaction failure and checking supported transfers. It makes model predictions, current network checks and later execution outcomes available as separate observations.

7.5 Limitations

The implemented system provides an inspectable ML comparison, current balance and simulation checks, and a record linking decisions to verified receipts. This is useful for studying failure risk and diagnosing supported transfer problems before submission.

The historical sample is sparse and originates from one provider. Inclusion timestamps approximate information availability rather than submission time. The selected table excludes unlanded attempts and full instruction payloads. Historical identity controls and participation features have slightly different update schedules. Live RPC sampling differs from training, and a simulation pass is state-dependent. Measured ML-driven fee savings and retry-time reductions are not established by the current experiments.

7.6 Scope for Future Work

Further work can evaluate independent data sources and periods, align identity-learning update schedules, and test cold-specific calibration under changing conditions. A prospective study should compare candidates that pass normal preflight before measuring any incremental ML benefit. Broader transaction types, authorised wallet integration and measured user task times are additional extensions. These are extensions to the completed research prototype rather than results claimed in this report.

REFERENCES

[1] Solana Foundation, "Accounts," official documentation. https://solana.com/docs/core/accounts. Accessed October 16, 2026.

[2] Solana Foundation, "Transactions," official documentation. https://solana.com/docs/core/transactions. Accessed October 16, 2026.

[3] X. Zheng, Z. Wan, D. Lo, D. Xie and X. Yang, "Why Does My Transaction Fail? A First Look at Failed Transactions on the Solana Blockchain," 2025. https://arxiv.org/abs/2504.18055. DOI: 10.1145/3728943.

[4] Solana Foundation, "Fees," official documentation. https://solana.com/docs/core/fees.

[5] Solana Foundation, "simulateTransaction," RPC documentation. https://solana.com/docs/rpc/http/simulatetransaction.

[6] Solana Foundation, "sendTransaction," RPC documentation. https://solana.com/docs/rpc/http/sendtransaction.

[7] scikit-learn developers, "HistGradientBoostingClassifier," official API documentation. https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.HistGradientBoostingClassifier.html.

[8] SolArchive, "SolArchive transaction dataset," Hugging Face, CC BY 4.0. https://huggingface.co/datasets/solarchive/solarchive. Revision specified in Chapter 4.

[9] M. Lu et al., "BRIGHT – Graph Neural Networks in Real-Time Fraud Detection," 2022. https://arxiv.org/abs/2205.13084.

[10] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001. https://doi.org/10.1214/aos/1013203451.

[11] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," 2016. https://arxiv.org/abs/1603.02754.

[12] G. Ke et al., "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," *Advances in Neural Information Processing Systems*, vol. 30, 2017. https://proceedings.neurips.cc/paper_files/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html.

[13] A. Niculescu-Mizil and R. Caruana, “Predicting Good Probabilities With Supervised Learning,” in *Proceedings of ICML*, 2005. https://doi.org/10.1145/1102351.1102430.

[14] T. Saito and M. Rehmsmeier, “The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets,” *PLOS ONE*, vol. 10, no. 3, e0118432, 2015. https://doi.org/10.1371/journal.pone.0118432.

[15] S. Kapoor and A. Narayanan, “Leakage and the Reproducibility Crisis in ML-based Science,” arXiv:2207.07048, 2022. https://arxiv.org/abs/2207.07048.

[16] K. Weinberger, A. Dasgupta, J. Attenberg, J. Langford and A. Smola, “Feature Hashing for Large Scale Multitask Learning,” 2009. https://arxiv.org/abs/0902.2206.

[17] W. McKinney, “Data Structures for Statistical Computing in Python,” in *Proceedings of the 9th Python in Science Conference*, pp. 56–61, 2010. https://doi.org/10.25080/Majora-92bf1922-00a.

[18] C. R. Harris et al., “Array programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020. https://doi.org/10.1038/s41586-020-2649-2.

[19] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. https://jmlr.org/papers/v12/pedregosa11a.html.

[20] scikit-learn developers, “average_precision_score,” API documentation. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.average_precision_score.html. Accessed October 16, 2026.

[21] scikit-learn developers, “brier_score_loss,” API documentation. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.brier_score_loss.html. Accessed October 16, 2026.

[22] scikit-learn developers, “SGDClassifier,” API documentation. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.SGDClassifier.html. Accessed October 16, 2026.

[23] scikit-learn developers, “IsotonicRegression,” API documentation. https://scikit-learn.org/stable/modules/generated/sklearn.isotonic.IsotonicRegression.html. Accessed October 16, 2026.

[24] FastAPI contributors, “FastAPI,” official documentation. https://fastapi.tiangolo.com/. Accessed October 16, 2026.

[25] Solana Foundation, “getLatestBlockhash,” official documentation. https://solana.com/docs/rpc/http/getlatestblockhash. Accessed October 16, 2026.

[26] Solana Foundation, “getFeeForMessage,” official documentation. https://solana.com/docs/rpc/http/getfeeformessage. Accessed October 16, 2026.

[27] Solana Foundation, “getBalance,” official documentation. https://solana.com/docs/rpc/http/getbalance. Accessed October 16, 2026.

[28] Solana Foundation, “getBlock,” official documentation. https://solana.com/docs/rpc/http/getblock. Accessed October 16, 2026.

[29] Solana Foundation, “getTransaction,” official documentation. https://solana.com/docs/rpc/http/gettransaction. Accessed October 16, 2026.

[30] SQLite developers, “CREATE TRIGGER,” SQL language documentation. https://www.sqlite.org/lang_createtrigger.html. Accessed October 16, 2026.